

Salesforce Developer Bridge Program

Submitted By :

Name : Sudharshan Kuppili

Roll No : 24PA5A0517

Branch : Computer Science and Engineering (CSE)

Program : Salesforce Developer Bridge Program

Day_2

Hospital OPD Management System

Task 1: Apex Collections

In this task, I practiced Apex Collections by working with List, Set, and Map collections.

I created a List to store sample values and iterated through them using a loop.

I used a Set to store unique Patient record IDs and learned that duplicate values are automatically removed.

I also created a Map to store Patient records using their Id as the key and retrieved records efficiently.

I executed the code successfully using Execute Anonymous and verified the output in the Debug Log.

Learning Outcome:

I learned how to use List, Set, and Map collections in Apex to efficiently store, retrieve and manipulate data.

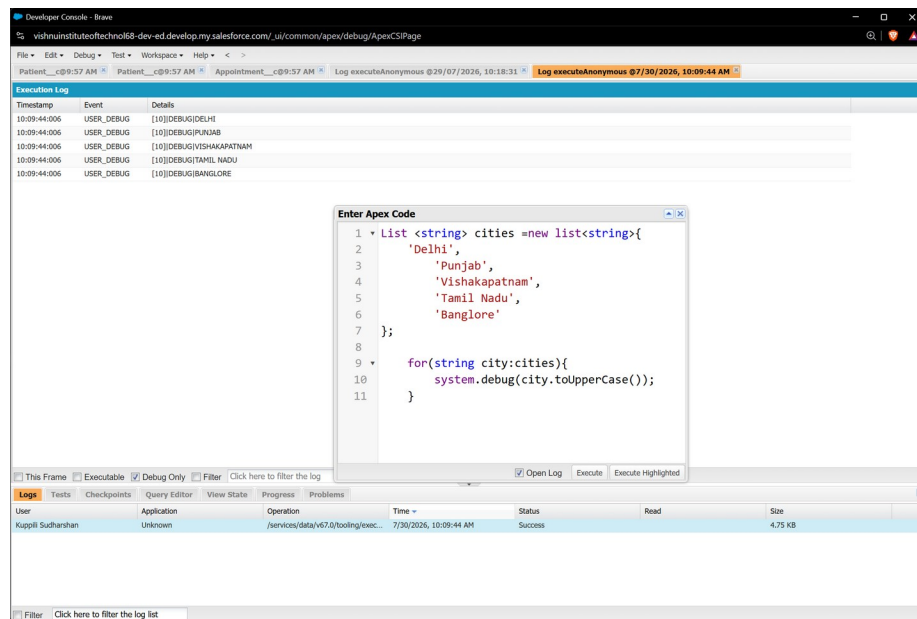


Figure 1: List of City Names – Execute Anonymous & Debug Log

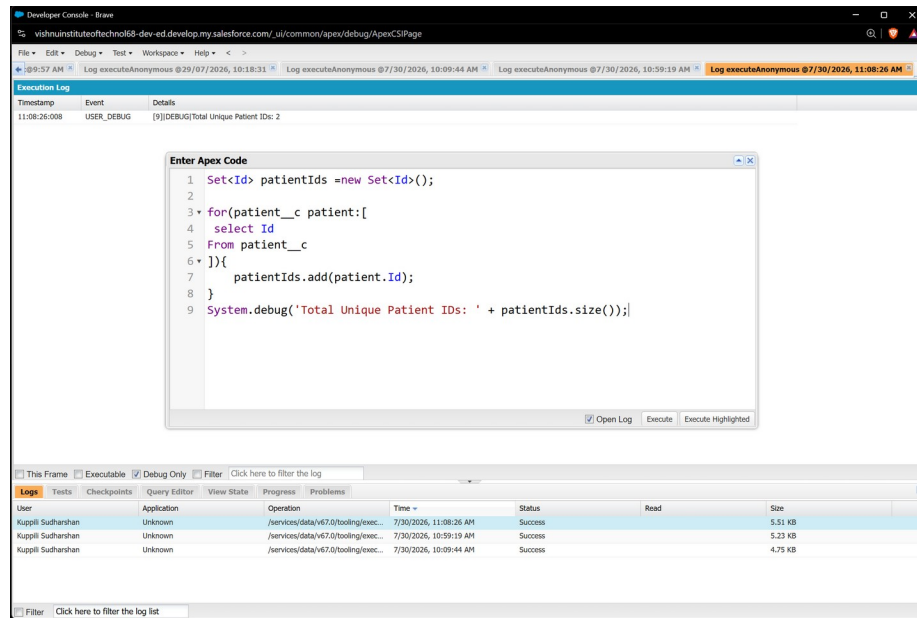


Figure 2: Set of Unique Patient IDs – Execute Anonymous & Debug Log

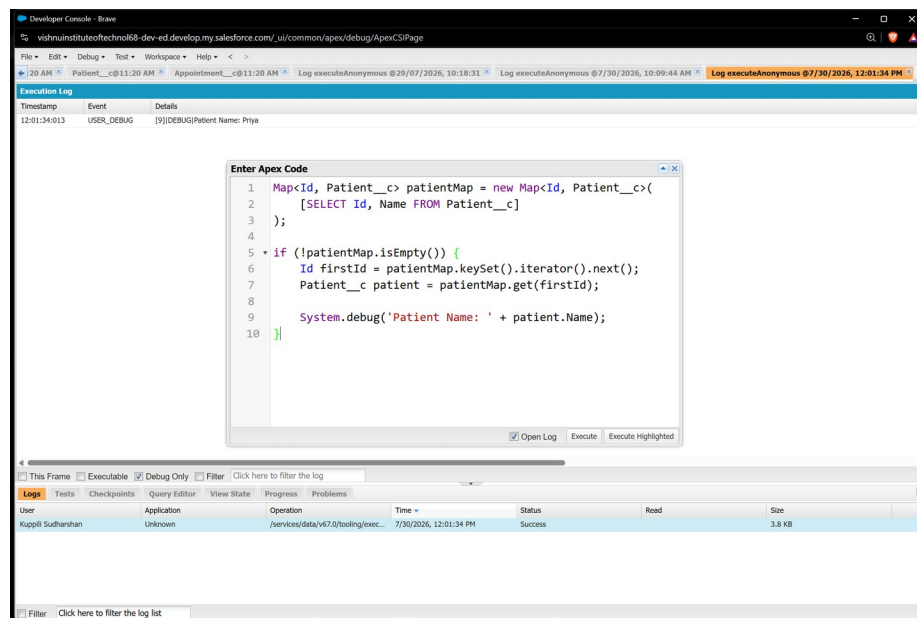


Figure 3: Map of Patient Records by Id – Execute Anonymous & Debug Log

Task 2: Governor Limits & Bulkification

In this task, I learned about Salesforce Governor Limits and why Apex code must be bulkified.

I understood why SOQL queries should never be placed inside loops.

I practiced bulkification by collecting record IDs using a Set and executing a single SOQL query outside the loop.

I also learned how bulkified code improves performance and follows Salesforce best practices.

Learning Outcome:

I learned how to write scalable Apex code by following Governor Limits and Bulkification best practices.

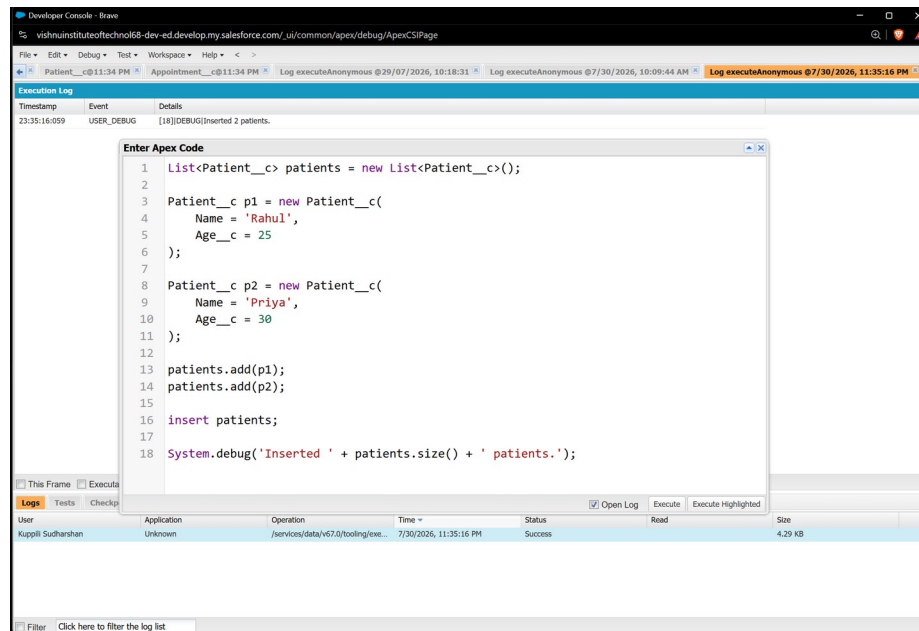


Figure 4: Bulk Insert of Multiple Patient Records in a Single DML Statement

Task 3: Trigger Handler Pattern

In this task, I implemented the Trigger Handler Pattern.

I created an Apex Trigger named AppointmentTrigger that executes before inserting Appointment records.

Instead of writing business logic inside the trigger, I called a separate handler class named AppointmentTriggerHandler.

This approach improves code readability, maintainability and follows Salesforce best practices.

```
trigger AppointmentTrigger on Appointment__c (before insert) {
    AppointmentTriggerHandler.preventDuplicateAppointments(Trigger.new);
}
```

I also wrote an AppointmentTriggerHandlerTest class and ran it in the Developer Console, which confirmed 100% code coverage on both AppointmentTrigger and AppointmentTriggerHandler.

Learning Outcome:

I learned how to separate business logic from Apex Triggers using the Trigger Handler Pattern.

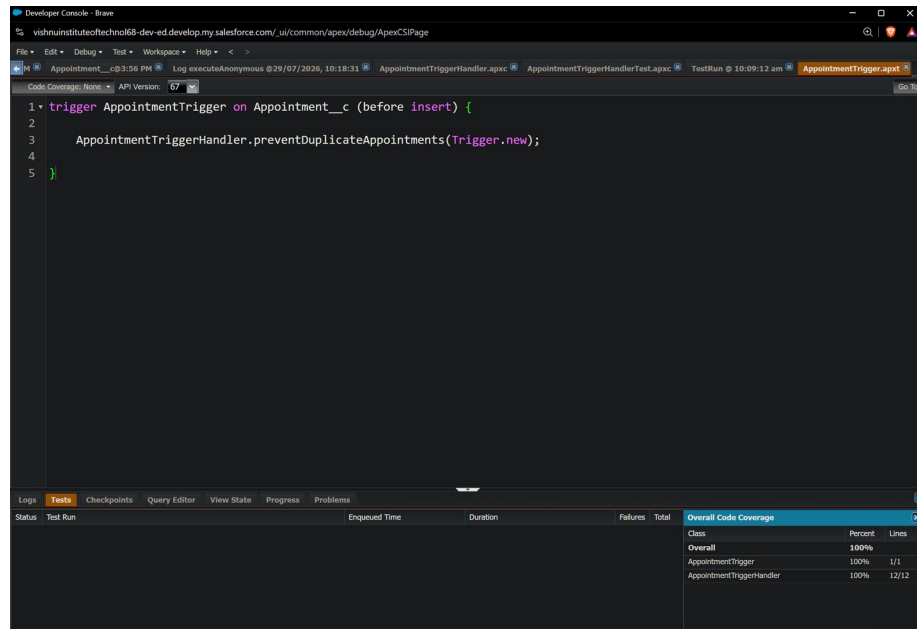


Figure 5: AppointmentTrigger Code – 100% Code Coverage on Trigger and Handler

Task 4: Duplicate Appointment Validation

I implemented duplicate appointment validation using the AppointmentTriggerHandler class.

The handler receives Trigger.new records.

It collects Patient IDs using a Set.

It queries existing Appointment records using SOQL.

It compares Patient and Appointment Date.

If a duplicate appointment exists, it prevents the record from being saved using addError().

This ensures that the same patient cannot have multiple appointments on the same date.

```
public class AppointmentTriggerHandler {

    public static void preventDuplicateAppointments(List<Appointment__c>
newAppointments){

        Set<Id> patientIds = new Set<Id>();

        for(Appointment__c app : newAppointments){
            if(app.Patient__c != null){
                patientIds.add(app.Patient__c);
            }
        }

        List<Appointment__c> existingAppointments = [
            SELECT Id, Patient__c, Appointment_Date__c
            FROM Appointment__c
            WHERE Patient__c IN :patientIds
        ];
```

```

        for(Appointment__c newApp : newAppointments){
            for(Appointment__c oldApp : existingAppointments){
                if(newApp.Patient__c == oldApp.Patient__c &&
                    newApp.Appointment_Date__c == oldApp.Appointment_Date__c){
                    newApp.addError('This patient already has an appointment on
this date.');
```

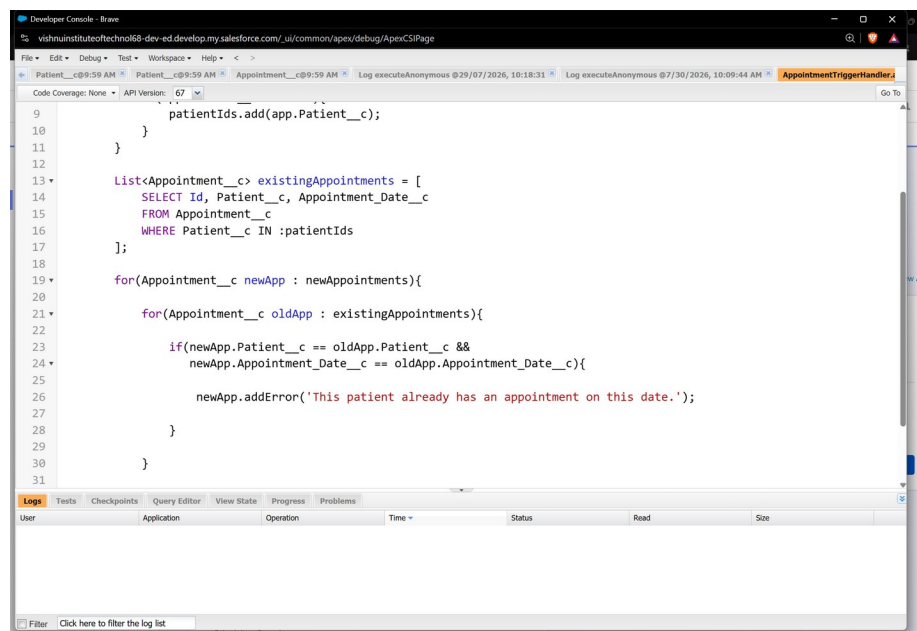


Figure 6: AppointmentTriggerHandler - Duplicate Check Logic

Learning Outcome:

I learned how to use SOQL, Trigger.new, Sets and addError() to implement business validation in Salesforce Apex.